CLAUDE CERTIFICATION PROGRAM · EXAM CCAR-F

Architect Workbook

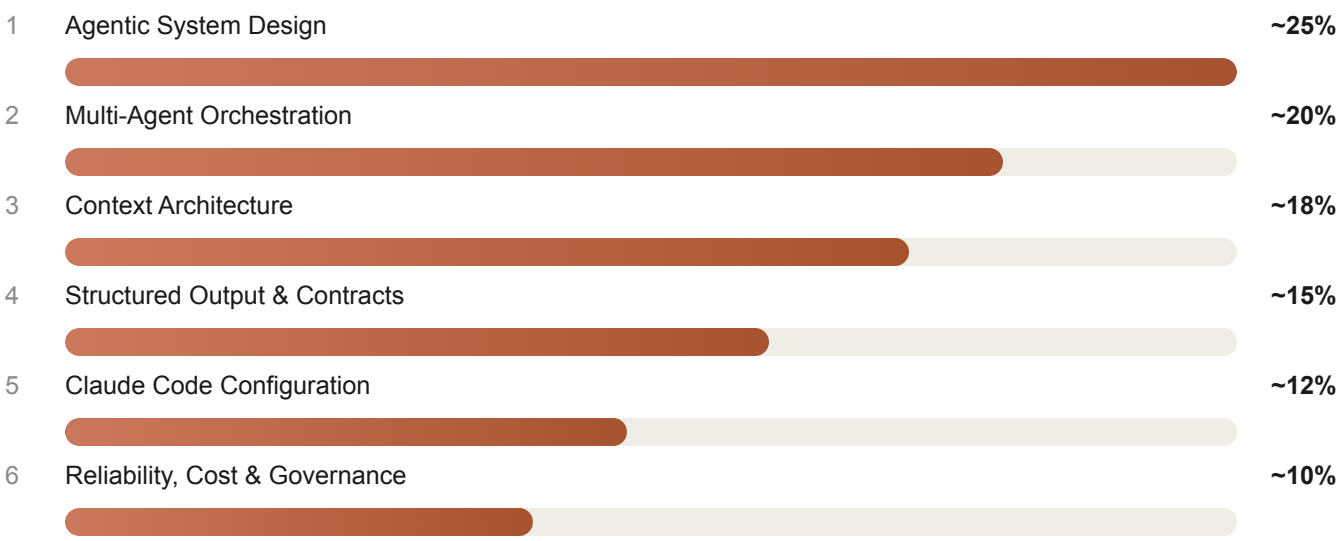
Claude Certified Architect — Foundations

22 modules across 6 domains · scenario-based, closed-book · design judgment over syntax

This credential validates *production-grade architectural judgment*: designing agentic systems, orchestrating multi-agent pipelines, configuring Claude Code, engineering reliable structured output, and managing context. The exam is scenario-based and closed-book — so this workbook trains the thing being tested: reading a situation, naming the binding constraint, choosing a topology, and defending the trade-off you accepted.

Domains	**6**	Modules	**22**
Design drills	**20**	Scenario items	**28 with rationale**
Format	**Scenario-based**	Materials	**Closed-book**
Assumes	**CCDV-F-level fluency**	Prep time	**3–6 weeks**

Domain structure — derived from the published scope statement, weights inferred (see facing page)



How to use this workbook

READ THIS FIRST — WHERE THE STRUCTURE CAME FROM

For the Associate and Developer exams there are published domain outlines. **For CCAR-F there is not one available here.** The six domains below are derived directly from the credential's own scope statement — designing agentic systems, orchestrating multi-agent pipelines, configuring Claude Code, engineering reliable structured output, and managing context, all under "production-grade architectural judgment." The **percentage weights are my estimate of relative emphasis, not published fact**, and the module count is a study structure rather than an official syllabus.

Treat the weights as a study-time hint and nothing more. **Verify the real blueprint, format, and logistics with Anthropic before you register.** The *content* here — the patterns, trade-offs, and failure modes — stands on its own regardless of how the exam apportions its questions.

CLOSED-BOOK AND SCENARIO-BASED CHANGES HOW YOU SHOULD STUDY

You cannot look anything up, and you will not be asked to recall a parameter name. You'll be given a situation — a team, a workload, a constraint, a failure — and asked which design is right and why. So: **stop memorizing syntax and start rehearsing judgment.** For every pattern in this workbook you should be able to say, without notes, *when it applies, what it costs, what it fails at, and what you would choose instead.* The drills are built to produce exactly that.

Every module has four parts:

1 · Objectives

The design decisions you must be able to make and defend.

2 · Architecture

Patterns with their trade-offs stated explicitly — what each buys, what each costs, and the failure mode that follows from choosing it.

3 · Design drill

A scenario to work on paper. No code. Output is a decision plus its justification — usually a short architecture decision record.

4 · Scenario items

Exam-style items with rationale on why each alternative loses.

THE FOUR QUESTIONS BEHIND ALMOST EVERY ARCHITECT ITEM

1. What is the binding constraint? (latency · cost · accuracy · reversibility · compliance — usually one dominates)
2. What is the simplest structure that satisfies it? (single call → workflow → agent → multi-agent, in that order)
3. What happens when a step fails? (a design with no answer here is not a production design)
4. Who is accountable for the irreversible action? (if the answer is "the model", the design is wrong)
Run these four on any scenario before reading the options and most distractors eliminate themselves.

PREREQUISITE

This workbook assumes CCDV-F-level fluency: the Messages API is stateless, tool results return in one user message, caching is a prefix match, batches halve cost for latency-tolerant work. It does not re-teach those. If any is unfamiliar, work the Developer Workbook first — architecture built on a wrong mental model of the API fails in ways no diagram will reveal.

Contents

The architect's decision frame

One page. If you internalize nothing else, internalize this — nearly every scenario item is one of these five decisions wearing a costume.

1 · The autonomy ladder — climb only as far as you must

RUNG	WHO DECIDES THE NEXT STEP	CHOOSE IT WHEN
Single call	Nobody — one shot	The task is one transformation: classify, extract, summarize, rewrite.
Workflow	**Your code**	The steps and success criteria are known in advance. Deterministic, debuggable, cheap.
Single agent	**The model**, within one context	The path can't be specified up front; the model must adapt based on what it finds.
Multi-agent	The model, across **isolated contexts**	Subtasks need genuine context isolation, parallelism, or independent verification.

Each rung buys adaptability and costs determinism, latency, spend, and debuggability. **Descending the ladder is almost always the right answer to a reliability problem.** An exam option offering more autonomy for a well-specified task is a distractor.

2 · Reversibility governs autonomy

Reversible

Draft, analysis, suggestion, sandbox write. **Automate freely.**

Costly to reverse

Ticket updates, non-prod deploys, bulk edits. **Gate or make idempotent.**

Irreversible

Sent mail, payments, prod writes, deletions, public posts. **Human commits.**

3 · The constraint triangle

Latency, cost, and accuracy trade against each other. You cannot optimize all three — **name which one binds**, optimize it, and state what you gave up. A design document that claims to improve all three simultaneously is hiding an assumption.

4 · Context is a budget, not a container

Every architecture decision spends context: tool schemas, system prompt, history, tool results, retrieved documents. In a stateless API you refill it on every call. Designs fail in production not because the model is weak but because the budget was never managed (Domain 3).

5 · Contracts at every boundary

Between a model and your code, between agents, between a pipeline stage and the next — define the shape, enforce it mechanically, and decide what happens when it's violated. "It usually returns valid JSON" is not a contract (Domain 4).

THE DELIVERABLE FORMAT USED THROUGHOUT THIS WORKBOOK

Most drills ask for a short **architecture decision record**. Keep it to five lines:
Context — the situation and the binding constraint · **Decision** — what you chose · **Alternatives** — what you rejected and why · **Consequences** — what this costs and what it makes harder · **Revisit when** — the signal that would change the decision.

> That last line is what separates an architect from someone with an opinion — and writing these is the single best preparation for a closed-book scenario exam.

D1 Agentic System Design

~25% · 5 modules

The largest domain. Its recurring question is not "can an agent do this?" but "should this be an agent, and what does it cost me when it's wrong?"

1.1 Choosing the Autonomy Level

~75 MIN

Objectives — Place a workload on the autonomy ladder and defend it; recognise over-engineering and under-engineering from a scenario description.

The four-criteria gate — all must hold before you build an agent

CRITERION	QUESTION	IF THE ANSWER IS NO
Complexity	Is the task multi-step and hard to fully specify in advance?	Use a workflow. You already know the steps.
Value	Does the outcome justify higher cost and latency?	Use a single call or a cheaper tier.
Viability	Is the model actually capable at this task type?	Don't build it yet. Prototype and evaluate first.
Cost of error	Can mistakes be caught and recovered from?	Add verification and human gates — or don't automate the decision at all.

Reading the scenario — signals and what they mean

SCENARIO LANGUAGE	POINTS TO
"the same four steps every time", "our documented process"	**Workflow.** Known sequence.
"depends what it finds", "may need to investigate further"	**Agent.** Path is data-dependent.
"must be auditable / reproducible / explained to a regulator"	**Workflow**, or an agent with a hard-gated commit step. Determinism is the requirement.
"thousands per hour", "sub-second"	**Single call**, small tier. An agent loop cannot meet that latency budget.
"we can't enumerate the cases in advance"	**Agent** — the honest signal for genuine adaptability.
"it must never do X"	Whatever the tier: enforce X in the **harness**, not the prompt.

THE TWO FAILURE DIRECTIONS, AND WHICH ONE THE EXAM PUNISHES MORE

Over-engineering — an agent where a workflow suffices. Symptoms: nondeterministic output on a deterministic task, cost 10× the alternative, unexplainable behavior, hard to test.

Under-engineering — a rigid workflow for genuinely open-ended work. Symptoms: an ever-growing thicket of special-case branches, and a pipeline that fails on inputs nobody anticipated.

Both are wrong, but scenario exams punish over-engineering more often, because it's the more tempting mistake — the more sophisticated design *feels* like the better answer.

DESIGN DRILL 1.1 — PLACE FIVE WORKLOADS

For each, choose a rung and write a one-line justification plus the constraint that decided it:

1. Classify 40,000 support emails/day into 12 categories.
2. Given a bug report, reproduce it, find the cause, propose a patch.
3. Every night, generate an exec summary from three fixed dashboards.
4. Answer questions about a 900-page policy manual for internal staff.
5. Given a vague feature request, produce a spec, get sign-off, then implement.

Then the real exercise: for each, name the signal that would make you climb one rung, and the signal that would make you descend. If you can't name both, you chose by instinct rather than by constraint.

1.2 Task Decomposition & Boundaries

~60 MIN

Objectives — Cut a large task into steps with verifiable boundaries; decide what the model owns versus what code owns.

THE DECOMPOSITION RULE

Give the model the judgment; give your code the determinism. Anything checkable, countable, ordered, or transactional belongs in code. Language understanding, ambiguity resolution, and summarization belong to the model. A step that mixes both is a step you can't test.

GIVE TO THE MODEL	KEEP IN CODE
Interpreting unstructured input	Arithmetic, totals, reconciliation
Classification and routing decisions	Executing the route
Drafting and summarizing	Persistence, transactions, idempotency
Choosing which tool fits	Authorization for that tool
Resolving ambiguity	Enforcing invariants and limits

A well-cut step has four properties

1. **A single deliverable** — you can name what comes out.
2. **A verifiable done-condition** — you can check it without asking the model.
3. **An owner** — model or code, not both.
4. **A defined failure behavior** — retry, escalate, or abort. Steps without this are where pipelines hang in production.

DESIGN DRILL 1.2 — CUT ONE APART

Take "process an incoming vendor invoice end to end." Produce 8–12 steps, each labelled `model` or `code`, with its done-condition and failure behavior. Then mark every step that touches money or writes to a system of record — those are your gate candidates for Module 1.4. Most people's first cut has the model doing arithmetic somewhere; find it.

1.3 Tool Surface Architecture

~75 MIN

> **Objectives —** Design a tool surface that is capable enough to do the job and narrow enough to be governable.

The tool surface **is** the agent's capability boundary. It determines what the agent can do, what your harness can intercept, and the blast radius when something goes wrong. This is the highest-leverage architectural decision in Domain 1.

DESIGN	WHAT IT BUYS	WHAT IT COSTS
One broad tool (bash / exec)	Maximum reach; the agent can do almost anything.	The harness sees an opaque string. You cannot gate, render, audit, or parallelize per-action. Everything is one permission.
Dedicated typed tools	Typed arguments the harness can inspect, gate, log, and parallelize. Per-action permissions.	More surface to design and maintain; risk of an over-large tool set diluting selection.
Hybrid (recommended)	Bash for breadth, promoted tools for anything gated, rendered, audited, or parallel-safe.	Requires an explicit promotion rule so the surface doesn't sprawl.

THE PROMOTION RULE

Promote an action from bash to a dedicated tool when it needs to be **gated** (hard to reverse), **rendered** (needs UI), **audited** (compliance), or **parallelized** (read-only and safe to run concurrently). A `send_email` tool can be gated; `bash -c "curl -X POST …"` cannot — the harness has no idea what that string does.

TWO SURFACE-DESIGN FAILURE MODES

Too broad: one tool, one permission, unbounded blast radius — and no way to require approval for the dangerous 5% without blocking the safe 95%.

Too many: 40 overlapping tools dilute selection and bloat every request's context (tool schemas render first and are part of the cache prefix — Domain 3). If the surface must be large, use tool search so schemas load on demand rather than all at once.

1.4 Human-in-the-Loop & Reversibility

~60 MIN

> **Objectives —** Place checkpoints where they matter; design gates that actually function as controls.

THE PLACEMENT RULE

Automate generation. Gate commitment. The checkpoint belongs immediately before the action becomes irreversible, externally visible, or costly — not at the start (where there's nothing to review) and not at the end (where it already happened).

What makes a gate a control rather than a formality

REQUIREMENT	WHY IT FAILS WITHOUT IT
Competent reviewer	Someone who can't evaluate the output can only rubber-stamp it.
Sufficient information	Approving a diff you can't see is theater. Show what changes.
Realistic time budget	Thirty approvals in two minutes is not review — approval rate near 100% at high volume is the tell.
Genuine authority to reject	A gate that can't stop the pipeline isn't a gate.
Enforced by the harness	A prompt instruction to "ask before deleting" is guidance the model weighs, not a control it cannot bypass.

MODEL CONFIDENCE IS NOT A GATING SIGNAL

"Auto-approve when the model is confident" fails precisely in the cases the gate exists for — a confident hallucination scores high. If you need a risk-based gate, derive it from **properties of the action** (amount, recipient domain, blast radius, whether it's reversible), never from the model's self-report. This is one of the most reliably-tested judgments in the whole credential.

REDUCE GATE FATIGUE WITHOUT REMOVING THE GATE

If reviewers face too many approvals, don't loosen the gate — **reduce what needs gating**. Batch related approvals into one decision, make more actions reversible (write to a staging area, use drafts rather than sends), or narrow the tool so the risky path is rare. A gate everyone clicks through has the cost of a control and the value of none.

1.5 Failure Modes of Agentic Systems

~75 MIN

Objectives — Name the characteristic failure modes and their structural mitigations — the diagnosis half of Domain 1.

FAILURE MODE	WHAT IT LOOKS LIKE	STRUCTURAL MITIGATION
Runaway loop	Agent calls tools indefinitely; cost climbs with no progress.	Hard iteration cap · budget ceiling · progress check that aborts when state stops changing.
Goal drift	Ends up solving an adjacent problem; plausible but not what was asked.	Restate the goal in-loop · verify against the original spec · a separate evaluator.
Cascading error	An early wrong fact is treated as ground truth by every later step.	Validate at step boundaries · make steps independently checkable · don't let step N invent inputs for N+1.
Context exhaustion	Long run degrades, contradicts itself, or truncates.	Domain 3 in full — budgeting, compaction, editing, memory.
Silent tool failure	A tool errors; the agent proceeds on an empty result.	Return explicit errors (never empty-on-failure) · check server-tool result blocks · fail loudly.
Injection-driven action	Content the agent reads redirects what it does.	Least privilege · gate commitments · credentials outside model-readable context.
Confident fabrication	Invented API, citation, or record, delivered fluently.	Ground in retrieval · require sources · verify before acting, not after.
Nondeterminism in an audited path	Two runs, two different justifications for the same decision.	Move the decision into code; let the model produce evidence, not the verdict.

THE ARCHITECT'S REFLEX

For every failure mode, the durable fix is **structural** — a cap, a gate, a contract, a narrowed permission, a verification step. Prompt wording helps and should be used, but a system whose only defense is a well-worded instruction has no defense at all. When a scenario offers "improve the prompt" against "add a structural control," the structural control wins on any consequential path.

DESIGN DRILL 1.5 — PRE-MORTEM

Take a design from drill 1.1 and assume it failed badly in production six months from now. Write the incident summary: what failed, what it cost, what the first signal was, and which structural control would have caught it. Do this for three of the modes above. **Then add the control to the design.** This drill is the closest thing in the workbook to what the exam actually asks you to do.

Domain 1 — scenario items

ITEM A1-1 · MODULE 1.1

A compliance team processes regulatory filings through a documented, unchanging six-step procedure. Every decision must be reproducible and explainable to an auditor. They ask you to design an autonomous agent. What do you recommend?

A. An agent with tools for each step, so it can adapt to unusual filings

B. A deterministic workflow with Claude calls at the steps needing language understanding, and decisions in code

C. A multi-agent pipeline with a specialist per step

D. A single agent with a detailed system prompt describing the six steps

B. The procedure is known and the binding constraint is *reproducibility* — which model-driven control flow directly undermines. A, C, and D all introduce nondeterminism into a path that must be explainable to an auditor; D is the subtlest wrong answer because describing the steps in a prompt looks like specifying them, but the model still chooses at runtime. Note the request itself was for an agent — architects are expected to push back when the ask conflicts with the constraint.

ITEM A1-2 · MODULE 1.4

An agent files expense reports. Reports under $50 post automatically; larger ones queue for approval. Reviewers approve ~200/day and approval rate is 99.4%. What's the architectural concern?

A. The $50 threshold is too low

B. The gate has become a formality — at that volume and approval rate, no real evaluation is occurring

C. The agent should assess its own confidence to reduce reviewer load

D. No concern — a human approves every large report

B. A 99.4% approval rate at 200/day is the classic signature of rubber-stamping: the control exists on the diagram but performs no evaluation. The fix is to *reduce what needs gating* — risk-based routing on action properties, batching, better summaries for review — not to loosen it. C substitutes the model's self-report for judgment, which fails exactly on the cases that matter. D mistakes the presence of a review step for the presence of review.

ITEM A1-3 · MODULE 1.5

A research agent occasionally runs for hours, making hundreds of tool calls without converging, producing large bills. Most robust mitigation?

A. Add "be efficient and don't overthink" to the system prompt

B. Switch to a faster model

C. Enforce a hard iteration cap and a token budget ceiling, with a progress check that aborts when state stops changing

D. Increase `max_tokens` so it finishes sooner

E. Run it as a batch job

C. Runaway loops need enforced bounds, not persuasion. A is guidance the model weighs against everything else in context; B and D change speed and per-response length while leaving the unbounded loop intact; E relocates the same runaway to a different execution mode. The progress check matters as much as the cap — an agent making steady progress toward a hard problem is fine, one spinning on the same state is not.

ITEM A1-4 · MODULE 1.3

An agent needs broad filesystem and shell access for a migration task, but writes to the production config directory must always require approval. Best design?

A. A single bash tool, with a prompt instruction to ask before touching production config

B. A single bash tool, with a post-hoc audit log reviewed weekly

C. Bash for general work, plus a dedicated write tool for the protected path that the harness gates — and deny the protected path in bash

D. Remove bash and enumerate every possible file operation as a separate tool

C. This is the promotion rule applied precisely: keep breadth where breadth is needed, promote the one action requiring a gate so the harness can intercept it, and close the bypass. A relies on prompt compliance for a hard guarantee. B detects damage a week late. D throws away the breadth the task actually requires, and an incomplete enumeration just fails differently.

D2 Multi-Agent Orchestration

~20% · 5 modules

The domain where sophistication is most tempting and most often wrong. Multi-agent is a real tool with real costs — the architect's job is knowing which.

2.1 When Multi-Agent Actually Wins

~75 MIN

Objectives — Justify a multi-agent design on grounds other than elegance; recognise when one agent with good tools is better.

THE THREE LEGITIMATE REASONS — AND THERE ARE ONLY THREE

1. Context isolation. A subtask would flood the main context with detail nobody needs afterwards (read 40 files, return one answer). The subagent burns its own context; the parent gets the conclusion.

2. Parallelism. Genuinely independent subtasks that would otherwise run serially — wall-clock is the constraint.

3. Independent verification. A reviewer with a *fresh* context catches what the author cannot, because it doesn't share the author's assumptions.

"Specialization" is usually **not** a fourth reason — a single agent with clear instructions and the right tools typically handles multiple roles fine. If your only argument is that each agent has a different persona, you probably want one agent.

WHAT YOU GAIN	WHAT YOU PAY
Context isolation per subtask	Handoffs lose fidelity — the parent sees a summary, not the evidence
Wall-clock parallelism	Token cost multiplies; each agent re-establishes its own context
Independent verification	Coordination complexity, partial-failure states, harder debugging
Specialized tool surfaces	More moving parts to secure, gate, and observe

THE UNDER-EXAMINED COST: DEBUGGABILITY

When a single agent misbehaves you read one transcript. When a five-agent pipeline produces a wrong answer you must work out *which* agent was wrong, whether it was wrong because of a bad handoff, and whether the parent misinterpreted a correct result. Multi-agent systems are meaningfully harder to debug, and that cost is paid every day of operation — not just at design time.

2.2 Topologies

~90 MIN

Objectives — Name each topology, its shape, and its characteristic failure — then select one from a scenario.

TOPOLOGY	SHAPE	USE WHEN	CHARACTERISTIC FAILURE
Pipeline (chaining)	A → B → C, fixed order.	Stages are known and sequential; each transforms the last.	Error cascade — a wrong step 1 corrupts everything downstream, confidently.
Router	Classify, then dispatch to a specialist.	Distinct input classes need different handling or different cost tiers.	Misclassification sends work down a path that cannot recover it.
Parallel fan-out	Split → N workers concurrently → merge.	Independent subtasks; wall-clock matters.	Merge conflicts; partial failure leaves an ambiguous aggregate.
Orchestrator-workers	A lead decomposes dynamically and delegates.	Subtasks aren't known until runtime.	Orchestrator over-delegates trivia; coordination cost exceeds the work.
Evaluator-optimizer	Generate → critique → revise, looping.	Quality bar is expressible as criteria and first drafts reliably miss it.	Infinite polish loop; a lenient evaluator that passes anything.
Voting / consensus	Same task N times, agree on the answer.	High-stakes classification where errors are expensive and independent.	N× cost; correlated errors mean agreement isn't independence.

EVALUATOR-OPTIMIZER: USE A SEPARATE CONTEXT

Asking one agent to critique its own output produces mostly agreement — it shares every assumption that produced the flaw. A **separate evaluator with a fresh context and an explicit rubric** is materially more effective. Always bound the loop (max revisions) and define what happens when it exhausts them: ship the best attempt, escalate to a human, or fail. That last decision is the one people forget.

DESIGN DRILL 2.2 — MATCH TOPOLOGY TO SCENARIO

Choose a topology and name its characteristic failure plus your mitigation:

1. Translate a product catalogue into 12 languages, each independent.
2. Draft a legal summary that must meet a firm's written quality standard.
3. Triage inbound security alerts: most are noise, some need deep investigation.
4. Given a research question, find sources, extract claims, check consistency, synthesize.
5. Decide whether a transaction is fraudulent; false negatives are very expensive.

Then: for two of them, argue the case that a *single* agent would be the better choice. If that argument is easy to make, the multi-agent design was decoration.

2.3 Handoff Contracts & Context Isolation

~75 MIN

Objectives — Design what passes between agents; decide what each agent may and may not see.

THE HANDOFF IS THE ARCHITECTURE

In a multi-agent system, **the interfaces matter more than the agents**. An agent whose input is a vague paragraph will fail unpredictably no matter how good it is. Specify each handoff like an API: what's in it, what shape it takes, what's guaranteed, and what happens when it's malformed.

HANDOFF ELEMENT	DESIGN QUESTION
Payload shape	Structured (schema-enforced) or prose? Between agents, prefer structured — see Domain 4.
Provenance	Does the receiver need to know where facts came from, or just the conclusion? Losing provenance is how unverifiable claims propagate.
Confidence / gaps	Can the sender say "I couldn't determine X"? Without an explicit slot for uncertainty, agents fill gaps with plausible invention.
Scope	What must the receiver *not* see — for context economy, and for privilege separation?
Failure signal	How does the sender report that it failed, distinguishably from returning a poor result?

THE LOSSY-HANDOFF FAILURE

Subagent reads 40 files and reports "the auth module looks fine." The parent takes that as verified fact and proceeds. But the summary dropped the caveat, the parent can't see the evidence, and no one can reconstruct the reasoning. **Design handoffs to carry the caveats, not just the conclusion** — an explicit field for "what I could not determine" is one of the highest-value fields in any multi-agent contract, and it costs almost nothing.

ISOLATION IS A PRIVILEGE BOUNDARY TOO

Different agents can have different tool surfaces and different credentials. A summarizer that reads untrusted external content should **not** hold the tool that sends email — then a successful prompt injection in the content it reads has nowhere to go. Splitting read-untrusted from act-externally is one of the strongest architectural mitigations available, and it's a Domain 1/6 idea that lands hardest here.

2.4 Coordination, State & Partial Failure

~75 MIN

Objectives — Decide where shared state lives; design for partial failure rather than assuming success.

STATE APPROACH	GOOD FOR	WATCH OUT FOR
Pass-through (state in messages)	Short pipelines; simple payloads.	Grows with every hop; each agent re-reads everything.
Shared workspace (files)	Agents collaborating on artifacts.	Concurrent writes; no transactionality; needs conventions.
External store (DB/queue)	Durable, resumable, auditable pipelines.	Most engineering; the right answer for anything long-running.
Orchestrator-held	Fan-out/fan-in where the lead owns the merge.	Orchestrator context becomes the bottleneck.

Partial failure — the questions an exam scenario is really asking

1. **One of five parallel workers fails — what does the merge produce?** A partial result labelled as partial, a retry, or an abort. Silently merging four of five is the bug.
2. **A pipeline stage fails at step 4 of 6 — can you resume?** Only if intermediate state was durable. Otherwise you re-run everything, at full cost.
3. **Are retries safe?** If a stage has side effects, a retry duplicates them unless it's idempotent.
4. **Is there a global budget?** Per-agent caps still multiply. Cap the *system*, not just each participant.

2.5 The Economics of Multi-Agent

~60 MIN

Objectives — Estimate the cost and latency profile of a topology before building it.

MULTI-AGENT MULTIPLIES TOKEN SPEND, AND NOT LINEARLY

Each agent carries its own system prompt, its own tool schemas, and its own accumulated context. Five agents is not 5× one agent's *useful* work — a good deal of it is re-establishing context that the parent already had. Orchestrator-worker designs also pay for the orchestrator's own reasoning about delegation. Budget accordingly, and be suspicious of a design whose justification is elegance.

TOPOLOGY	COST PROFILE	LATENCY PROFILE
Single agent	Baseline	Serial; bounded by the task.
Pipeline	Sum of stages; each re-establishes context	Serial — the sum of every stage. Worst case for wall-clock.
Parallel fan-out	N× worker cost + merge	**Best case** — roughly the slowest worker.
Orchestrator-workers	N× + orchestrator overhead	Partly parallel; the orchestrator serializes decisions.
Evaluator-optimizer	(generate + evaluate) × iterations	Serial and unbounded unless capped.
Voting	**N× per decision**	Parallel; justified only by error cost.

COST LEVERS STILL APPLY — AND MATTER MORE HERE

Right-size each agent independently (a summarizer subagent rarely needs the largest tier), keep each agent's prefix stable so caching works, and route latency-tolerant stages to batch processing. A multi-agent design where every participant runs the biggest model is the most common way these systems become indefensibly expensive.

DESIGN DRILL 2.5 — COST A TOPOLOGY BEFORE YOU BUILD IT

Take drill 2.2's item 4 (research → extract → check → synthesize). Estimate for both a single agent and a four-agent pipeline: total tokens, wall-clock, per-participant model tier, and what happens if stage 3 fails. Then write the ADR — and make the "Revisit when" line concrete: what measured signal would make you collapse the pipeline back into one agent?

Domain 2 — scenario items

ITEM A2-1 · MODULE 2.1

A team proposes five agents — researcher, writer, editor, fact-checker, formatter — for a content pipeline. Each has the same tools and reads the same context. What's your assessment?

A. Sound — specialization improves each stage

B. Likely over-engineered: identical tools and shared context mean none of the three real justifications (isolation, parallelism, independent verification) applies to most stages

C. Add a sixth coordinating agent

D. Sound, but they should all use the largest model

B. Same tools plus same context means no isolation and no parallelism — the "specialization" is just five system prompts, which one agent could switch between at a fraction of the cost. The one stage with a genuine claim is **fact-checking**, which benefits from a fresh context that doesn't share the writer's assumptions. C compounds the problem; D compounds the bill.

ITEM A2-2 · MODULE 2.3

In an orchestrator-worker system, a worker analyzes 60 log files and reports "no anomalies found." The orchestrator reports the system healthy. It wasn't. What's the design flaw?

A. The worker should have used a more capable model

B. The handoff carries a conclusion with no provenance, coverage, or uncertainty — the orchestrator cannot distinguish "checked thoroughly, found nothing" from "couldn't parse the files"

C. The orchestrator should re-read all 60 files itself

D. Workers shouldn't analyze logs

B. The contract is the flaw. A handoff carrying what was examined, what was skipped, and what couldn't be determined turns an unverifiable claim into a checkable one — and costs almost nothing to add. C defeats the entire purpose of context isolation. A might improve accuracy marginally while leaving the orchestrator equally unable to tell a thorough pass from a failed one.

ITEM A2-3 · MODULE 2.2

Generated marketing copy must meet a documented brand standard. First drafts meet it about 40% of the time. Reviewers are the bottleneck. Best topology?

A. Voting — generate three drafts and pick the most common

B. Evaluator-optimizer: a separate evaluator scores against the standard as an explicit rubric and returns specific revisions, bounded by a max-iteration cap

C. A longer system prompt describing the brand standard

D. A pipeline of three writers each improving the last

B. There's an articulable quality bar and drafts reliably miss it — the exact evaluator-optimizer profile, and it directly relieves the human bottleneck. The separate context matters: a self-critiquing writer mostly agrees with itself. A votes on *popularity*, not conformance to a standard. C is worth doing anyway but doesn't add the missing feedback loop. D has no evaluation step — three writers guessing in sequence.

ITEM A2-4 · MODULE 2.3

A summarization agent reads untrusted user-submitted documents. The same agent can send email. Strongest architectural mitigation?

A. Instruct the agent to ignore instructions found inside documents

B. Split into two agents: one reads untrusted content with no outbound tools, one sends email and never reads untrusted content — connected by a structured handoff

C. Scan documents for injection patterns before processing

D. Log all outbound email for later review

B. Privilege separation: an injection in the read agent has no outbound capability to hijack, so it cannot be turned into an action. A is a useful layer but is text competing with text. C is an unbounded blocklist against an attacker who can rephrase. D detects exfiltration after it happened. Only B makes the attack structurally unable to reach its objective.

ITEM A2-5 · MODULE 2.4

A fan-out design dispatches 8 parallel workers; results are merged into one report. In testing, one worker occasionally times out. Current code merges whatever returned. What should change?

A. Nothing — 7 of 8 is close enough

B. The merge must distinguish complete from partial results, and either retry the failed worker or label the output as partial with the gap named

C. Increase the timeout until failures stop

D. Run the workers serially instead

B. Silently merging 7 of 8 produces a report that *looks* complete and isn't — the reader has no way to know something is missing, which is worse than a visible failure. A accepts that. C reduces frequency without handling the case, and there's always a timeout you can exceed. D discards the parallelism that motivated the design and still doesn't answer what happens on failure.

D3 Context Architecture

~18% · 4 modules

Named explicitly in the credential's scope. Context is the resource that long-running agentic systems actually run out of — and managing it is a design decision, not a runtime patch.

3.1 The Context Budget

~60 MIN

Objectives — Account for everything competing for the window; design a budget before the system exists.

CONSUMER	GROWS WITH	ARCHITECTURAL LEVER
Tool schemas	Tool count	Narrow the surface; tool search for large libraries. Renders first — part of the cache prefix.
System prompt	Instruction sprawl	Keep durable rules only; move task specifics to the turn. Keep it byte-stable.
Conversation history	Turns	Compaction, summarization, retrieval.
Tool results	Agent iterations	**Usually the dominant term in an agent loop.** Context editing; return summaries not dumps.
Retrieved documents	Retrieval breadth	Tighter retrieval; rank and cut; cite rather than inline.
Thinking	Effort level	Effort tuning. Counts toward the output cap.

THE ARCHITECT'S FRAMING

A large context window is not permission to stop thinking about context. Every token you spend on history is one unavailable for the current problem — and in a stateless API you **re-send and re-pay for the whole thing on every call**. Context growth is therefore simultaneously a quality problem, a latency problem, and a cost problem. It is the same design decision seen from three angles.

THE SIGNATURE SYMPTOM, AND WHY TEAMS MISDIAGNOSE IT

A long-running agent that starts strong and degrades — contradicting earlier decisions, repeating work, losing the thread — is almost always a context problem, not a capability problem. Teams reliably respond by upgrading the model, which is expensive and doesn't help. **Diagnose context before capability** whenever quality degrades *over the course of a run* rather than being poor from the start.

3.2 The Four Levers

~90 MIN

Objectives — Distinguish the four mechanisms precisely and choose among them — this distinction is heavily testable because the options sound interchangeable and are not.

LEVER	MECHANISM	SCOPE	CHOOSE WHEN
Context editing	**Clears** stale tool results / thinking. Prunes; does not summarize.	Within a session	Bulk is old tool output that has served its purpose. The default lever for agent loops.
Compaction	**Summarizes** earlier context into a compaction block.	Within a session	The conversation itself is long and the gist must survive.
Memory	Reads/writes durable files.	**Across sessions**	State must outlive the process. The *only* lever that does.
Retrieval	Keeps corpus outside; injects relevant slices per turn.	External	The knowledge base is larger than any window, or changes independently.

THEY COMPOSE — AND LONG-RUNNING SYSTEMS USUALLY NEED THREE

A mature agent typically uses **context editing** to prune tool noise within a run, **compaction** when the dialogue itself grows long, and **memory** to carry conclusions into tomorrow's session, with **retrieval** supplying facts on demand throughout. These are not competing options; the exam question is which one solves the *specific* symptom described.

TWO DISTINCTIONS WORTH MEMORIZING VERBATIM

Editing vs. compaction: editing *deletes*, compaction *summarizes*. If the content must still be recoverable in substance, editing is the wrong lever.

Memory vs. everything else: memory is the only cross-session mechanism. "The agent should remember this next week" has exactly one answer — and it is never a bigger context window.

DESIGN DRILL 3.2 — DIAGNOSE THE SYMPTOM, PRESCRIBE THE LEVER

For each, name the lever (or combination) and justify it:

1. An agent reads 200 files in one run; by file 150 it has forgotten the task.
2. A support agent should recall a customer's preferences from a call three weeks ago.
3. A conversation spanning hours must retain decisions but not every exchange verbatim.
4. An assistant must answer from a 50,000-document policy corpus updated weekly.
5. A coding agent re-reads the same large file every few iterations.

Then: for each, state what the wrong lever would cost you. That's the discrimination the items test.

3.3 Caching as an Architectural Constraint

~75 MIN

Objectives — Treat prefix stability as a design property; recognise architectural choices that silently destroy it.

THE ARCHITECT'S VERSION OF THE CACHING RULE

Caching is a **prefix match**, and render order is `tools` → `system` → `messages`. Therefore: **prefix stability is an architectural property you design for, not an optimization you add later.** Once you see it that way, several attractive design choices reveal themselves as expensive.

DESIGN CHOICE	EFFECT ON CACHING
Per-user or per-tenant system prompt	**Destroys cross-user sharing.** Each tenant has a distinct prefix. Consider a shared prefix plus per-user content *after* the breakpoint.
Dynamic tool set per request	**Worst case** — tools render first, so changing them invalidates everything.
Timestamp or request id in the system prompt	Invalidates on every single request. The most common accidental cause.
Switching models mid-conversation	Caches are model-scoped — the new model starts cold.
Editing the system prompt mid-session	Invalidates the whole cached history. Prefer appending an operator instruction after the cached prefix.
Subagent with its own prompt/tools	Separate prefix, separate cache. Budget for the cold start of every subagent.

THE DESIGN PATTERN THAT FOLLOWS

Stable content first, volatile content last. Freeze the system prompt and the tool list; put per-request and per-user material in the message turns after the last breakpoint. In multi-agent designs, give each agent a stable prompt rather than composing one dynamically. This one ordering decision is often worth more than any other cost optimization in a high-volume system.

3.4 Long-Horizon Sessions & Handoff

~60 MIN

Objectives — Design systems that run for hours or resume across days.

1. **Externalize durable state early.** If a run must survive a restart, its state cannot live only in the conversation. Files, a database, or a memory store — decided at design time, not after the first crash.
2. **Checkpoint at meaningful boundaries.** Enough state to resume without redoing expensive work.
3. **Design the handoff summary deliberately.** Decisions made, constraints established, open questions, current state. This is what survives compaction — so write it as a first-class artifact rather than hoping the summary captures it.
4. **Re-anchor the goal periodically.** Long runs drift; restating the objective is cheap insurance.
5. **Bound the whole thing.** Wall-clock, token, and iteration ceilings at the *system* level.

DO NOT CONFUSE "RESUMABLE" WITH "LONG-RUNNING"

A resumable system can be killed and restarted without losing work — that requires durable external state. A long-running system merely runs a long time, and loses everything if the process dies. Scenarios describing overnight or multi-day work are usually asking for **resumability**, and an answer that only extends the session misses the requirement entirely.

Domain 3 — scenario items

ITEM A3-1 · MODULE 3.2

A code-review agent examines large repositories. Runs start well but by iteration 40 it contradicts earlier findings and revisits files it already reviewed. Best architectural response?

A. Upgrade to a more capable model

B. Apply context editing to clear stale tool results, and maintain a compact running findings list as durable state

C. Increase `max_tokens`

D. Split the repository and run separate independent agents

B. Degradation *over the course of a run* is the signature of context exhaustion — the file contents dominate the window and the task drifts out of focus. Editing removes the bulk; a running findings list preserves the conclusions that matter. A is the expensive reflex that doesn't address the cause. C affects output length, not input pressure. D is a plausible complement but doesn't fix a single agent's context management, and adds coordination cost.

ITEM A3-2 · MODULE 3.2

A customer-service agent should recall a customer's stated preferences from a conversation several weeks earlier. What does the architecture need?

A. A larger context window

B. Compaction enabled on the session

C. Cross-session persistence — a memory store or database, written at the end of each conversation and loaded at the start of the next

D. Longer session timeouts

C. Nothing within a session survives it — memory (or an equivalent external store) is the only cross-session mechanism. A and D both try to make one session cover weeks, which doesn't match how conversations actually arrive. B compresses *within* a session and vanishes with it. Note that the answer also implies a design decision about *what* to persist and for how long, which is where privacy review belongs.

ITEM A3-3 · MODULE 3.3

A multi-tenant assistant composes each request's system prompt from a shared base plus tenant-specific rules. At scale, input costs are far higher than projected. Most likely cause?

A. The model tier is too expensive

B. Per-tenant prompt composition gives every tenant a distinct prefix, so cache reuse is limited to that tenant's own repeat traffic

C. Streaming is disabled

D. Requests aren't batched

B. Caching is a byte-exact prefix match, so composing the prompt per tenant fragments the cache — a shared base that *looks* shared provides no shared caching once tenant rules are concatenated into it. The architectural fix is to keep the shared portion byte-identical and ahead of the breakpoint, with tenant rules after it. A and D are real levers but don't explain the overrun; C doesn't affect cost at all.

ITEM A3-4 · MODULE 3.4

A migration agent must run for several days, surviving restarts and deploys. Which property is essential?

A. A very large context window

B. Durable external state with checkpointing, so a restart resumes rather than restarts

C. Compaction enabled

D. A higher effort setting

B. Surviving a restart is the stated requirement, and no amount of in-session context management provides it — when the process dies, the conversation dies with it. A and C both extend how long one session can go without addressing durability; D affects reasoning depth. This is the long-running vs. resumable distinction, and it's the whole item.

D4 Structured Output & Contracts

~15% · 4 modules

"Engineering reliable structured output" is named in the credential's scope. At architect level this is about contracts at boundaries, not just getting valid JSON back once.

4.1 Designing the Output Contract

~75 MIN

Objectives — Design schemas that make failure visible; decide what the model may and may not omit.

DESIGN THE SCHEMA AROUND UNCERTAINTY, NOT JUST THE HAPPY PATH

The most consequential schema decision is **giving the model a legitimate way to say "I don't know."** A schema with a required `price` field and no null option guarantees an invented price when the source doesn't state one — the model must produce *something* valid. Add `"unknown"` enum values, nullable fields, and an explicit `gaps` or `not_found` array. This single habit prevents more downstream damage than any other schema practice.

DESIGN CHOICE	CONSEQUENCE
Required field with no null path	**Forces fabrication** when the data is absent.
Free-text field where an enum would do	Downstream string matching; inconsistent values across runs.
Deeply nested structure	Harder to fill correctly and harder to validate partially.
Confidence or evidence field	Lets consumers route low-confidence records to review rather than treating all output equally.
Flat, wide records	Easier to validate, diff, and store. Usually preferred.

ASK WHAT THE CONSUMER DOES WITH EACH FIELD

If nothing downstream branches on a field, it may not belong in the contract — every field is something the model must get right and something you must validate. Contracts should be as small as the consumer genuinely needs, and no smaller.

4.2 Enforcement Mechanisms

~60 MIN

> **Objectives —** Choose the enforcement level a boundary warrants; know what each mechanism does and does not guarantee.

MECHANISM	GUARANTEE	RIGHT BOUNDARY FOR IT
Schema-enforced output	The response conforms to your schema.	Machine-consumed output — the strongest and usual choice.
Strict tool arguments	Tool inputs validate exactly.	Anything the model triggers that has side effects.
Prompted format	**None.** Works until it doesn't.	Prototypes and human-read output only.
Post-hoc validation	Catches violations after generation.	Always — as the second layer, never as the only one.

SCHEMA CONFORMANCE IS NOT CORRECTNESS — THE ARCHITECT-LEVEL DISTINCTION

A schema guarantees **shape**, never **truth**. A perfectly valid record can contain a fabricated price, a mis-parsed date, or a confidently wrong classification. Schema enforcement eliminates parse failures; it does nothing about semantic error. Systems that treat schema-valid output as verified output have simply moved the failure from a visible crash to a silent bad record — which is worse. **Validate semantics separately**: range checks, referential integrity, cross-field consistency, and sampling against ground truth.

4.3 Validation & Graceful Degradation

~75 MIN

> **Objectives —** Design the failure path; decide what a violation does.

Every contract needs an answer to: **what happens when it's violated?** A design without one fails by whatever the code happens to do — usually an exception in the wrong layer, or a silent bad write.

STRATEGY	BEHAVIOR	APPROPRIATE WHEN
Reject	Fail the record; surface the error.	Correctness matters more than coverage. The safe default.
Retry	Re-request, ideally with the validation error fed back.	Transient formatting failures. Bound the attempts.
Partial accept	Keep valid fields; flag the rest.	Records are independently useful field by field.
Escalate	Route to human review.	Consequential records where neither dropping nor guessing is acceptable.
Fallback tier	Retry on a more capable model.	Systematic failures on hard inputs — measure before adopting.

NEVER SILENTLY DROP

The one universally wrong strategy is discarding invalid records without a trace. It produces a pipeline that looks healthy while its coverage quietly erodes — and the metric that would reveal it (records in vs. records out) is exactly the one nobody watches. **Every rejected record must be counted, and the rejection rate must be visible on a dashboard.** A rising rejection rate is one of the earliest and most reliable signals that something upstream has changed.

DESIGN DRILL 4.3 — SPECIFY A CONTRACT END TO END

Design the extraction contract for supplier invoices feeding an accounting system. Specify: the schema (with explicit unknown paths), the enforcement mechanism, the semantic validations, the failure strategy per validation class, and the two metrics you'd alarm on. Then answer the exam's favourite question: **what does the system do when a field is genuinely absent from the source document?** If your schema forces a value there, redesign it.

4.4 Contract Evolution

~45 MIN

Objectives — Change a contract without breaking consumers or invalidating history.

CHANGE	RISK	APPROACH
Add an optional field	Low	Safe. Consumers ignore what they don't know.
Add a required field	**Breaking**	Add as optional, backfill, then tighten.
Extend an enum	Medium	Consumers need a default branch *before* you ship the new value.
Remove or rename a field	**Breaking**	Deprecate, dual-write, migrate, then remove.
Tighten a constraint	Medium	Measure the would-be rejection rate against live traffic first.

THE PROMPT IS PART OF THE CONTRACT'S SURFACE

Changing the schema usually means changing the instructions and examples that produce it — and any change to a stable prefix invalidates the cache (Module 3.3). Version schema, prompt, and evaluation set **together**; re-run the eval before shipping. Treating the prompt as configuration you can tweak independently of the schema is how contracts silently drift out of sync with their consumers.

Domain 4 — scenario items

ITEM A4-1 · MODULE 4.1

An extraction pipeline pulls contract terms into a required schema. Auditors find fabricated values where the source document was silent. What's the root architectural cause?

A. The model is insufficiently capable

B. The schema requires every field with no representation for absent data, so producing valid output requires inventing values

C. Temperature is too high

D. The documents are too long

B. The schema itself made fabrication the only schema-valid behavior — the model is doing exactly what it was constrained to do. The fix is a legitimate absent path: nullable fields, an `"unknown"` enum, or an explicit gaps array. A treats a design error as a capability gap and would produce more *convincing* fabrications. This is the most important idea in Domain 4.

ITEM A4-2 · MODULE 4.2

A team enables schema-enforced output and reports "extraction is now reliable — every record validates." What should an architect flag?

A. Nothing — validation is the goal

B. Schema validity proves shape, not truth; semantic accuracy needs separate validation and sampling against ground truth

C. They should also lower the temperature

D. They should switch to prompted JSON for flexibility

B. A 100% validation rate says only that parse failures are gone. A fabricated but well-formed price validates perfectly, and the team has now lost the crash that used to surface bad input. Schema enforcement converts loud failures into silent ones unless you add semantic checks — which is precisely why the reported "reliability" is worth challenging.

ITEM A4-3 · MODULE 4.3

A nightly job processes 50,000 records; a small fraction fail validation and are skipped so the run completes. What's the architectural concern?

A. The job should stop entirely on the first failure

B. Silently dropping records erodes coverage invisibly — rejections must be counted, surfaced, and alarmed on when the rate moves

C. Validation should be removed to improve throughput

D. Records should be retried indefinitely

B. Skipping isn't inherently wrong — *silently* skipping is. Without a visible rejection rate, an upstream format change can quietly halve your coverage while every dashboard stays green. A is too brittle for bulk work; C removes the only thing catching bad data; D risks an unbounded loop on deterministically invalid input.

ITEM A4-4 · MODULE 4.4

A downstream team needs a new required field in an existing extraction contract. Safest rollout?

A. Add it as required and redeploy

B. Add it optional, backfill history, confirm consumers handle it, then tighten to required

C. Add it required and have consumers ignore missing values

D. Create a second parallel pipeline

B. The standard non-breaking evolution path: expand, migrate, then contract. A breaks every existing consumer and invalidates historical records at once. C is self-contradictory — a field consumers must tolerate as missing isn't required. D doubles cost and creates two sources of truth to reconcile.

D5 Claude Code Configuration

~12% · 2 modules

Named explicitly in the credential's scope. At architect level the question is governance: how a team configures Claude Code so behavior is consistent and constraints hold.

5.1 Project Configuration & the Instruction Hierarchy

~75 MIN

Objectives — Decide what belongs at each configuration layer; know which layers are advisory and which are enforced.

LAYER	NATURE	PUT HERE
Managed / org policy	**Enforced**	Organization-wide constraints individuals must not override.
Permission rules	**Enforced**	What may run unattended vs. what requires approval.
Hooks	**Enforced**	Deterministic actions on lifecycle events — blocking a command, running a formatter, logging for audit.
Project memory (`CLAUDE.md`)	Advisory	Conventions, architecture notes, commands, domain vocabulary. Checked into the repo, shared by the team.
Skills	Advisory	Packaged procedures loaded on demand when relevant.
Slash commands	Advisory	Reusable prompts invoked by name.
Session prompts	Advisory	Task-specific direction.

THE DISTINCTION THE EXAM IS BUILT ON

Advisory layers are read and weighed by the model. Enforced layers are executed by the harness and cannot be reasoned around. Anything phrased as "must never" belongs in an enforced layer. Writing "never run destructive commands" in `CLAUDE.md` is reasonable guidance and a *guarantee of nothing* — if the requirement is absolute, it belongs in a hook or a permission rule. Expect at least one item that offers a well-worded instruction against a harness control.

WHAT MAKES PROJECT MEMORY GOOD

Write for the model, not for onboarding humans. High value: build and test commands, directory layout, naming and style conventions, domain vocabulary, and *explicit non-goals* ("don't add new dependencies", "tests are offline-only"). Low value: prose the model can infer from the code itself. Keep it current — stale conventions produce confidently wrong output, and nothing signals staleness.

5.2 Extension & Team-Scale Governance

~75 MIN

Objectives — Choose the right extension mechanism; make a team's configuration consistent, reviewable, and safe.

MECHANISM	USE WHEN	NOT FOR
Project memory	Context every session needs.	Hard constraints; task specifics.
Skill	A recurring procedure worth packaging, loaded only when relevant.	Things needed on every single turn — that's memory.
Subagent	A scoped task deserving its own context — fan-out or fresh-eyes verification.	Work the main agent can do directly in one turn.
Hook	A constraint or side effect that must happen deterministically.	Guidance and preferences.
MCP server	Connecting an external system, reusable across clients.	Logic that belongs in your own code.
Slash command	A prompt people retype often.	Anything needing enforcement.

CONFIGURATION IS CODE — TREAT IT THAT WAY

Project memory, hooks, permission rules, and skills should be **version-controlled, code-reviewed, and rolled out deliberately**. A hook is executable logic on every developer's machine; an over-broad permission rule silently widens what runs unattended across the whole team. "It's just config" is how ungoverned changes reach every engineer at once.

TWO GOVERNANCE FAILURE MODES

Configuration drift — every engineer with their own local setup, so behavior varies per machine and bugs aren't reproducible. Fix by putting shared config in the repo.
Permission sprawl — allowlists that only ever grow, because each addition is individually reasonable. Review them on a schedule and remove what's unused; an allowlist nobody prunes eventually approximates "allow everything."

DESIGN DRILL 5.2 — CONFIGURE A TEAM

For a 20-engineer team adopting Claude Code on a production service, specify: what goes in project memory · which constraints become hooks or permission rules (and why each must be enforced rather than advised) · which recurring procedures become skills · what stays individual preference · and your review cadence for the permission list. **Then:** pick your single most important hook and describe what an engineer would experience when it fires. If that experience is confusing, the hook needs a better message.

Domain 5 — scenario items

ITEM A5-1 · MODULE 5.1

A regulated team requires that a specific data-export command *never* run without approval, even if a model is prompt-injected or an engineer asks for it directly. Where does this belong?

A. A strongly worded rule in `CLAUDE.md`

B. The system prompt for every session

C. A hook or permission rule enforced by the harness

D. A tool description warning about the risk

C. "Never, even under injection" is an absolute requirement, and only harness-enforced controls provide one. A, B, and D are all text the model reads and weighs against everything else in context — useful as guidance, worthless as a guarantee, and defeated by exactly the scenario described. The advisory/enforced split is the core of Domain 5.

ITEM A5-2 · MODULE 5.2

Engineers each maintain personal Claude Code configurations. Output quality varies widely and bugs are hard to reproduce. Best remedy?

A. Have everyone independently improve their own setup

B. Move shared context and constraints into version-controlled project configuration, reviewed like code, leaving genuine preferences individual

C. Standardize on one model for everyone

D. Stop using Claude Code on this project

B. The problem is configuration drift, so the fix is shared, reviewable configuration in the repo — which also makes behavior reproducible when something goes wrong. A entrenches the divergence. C addresses one variable among many. D discards the tool over a solvable governance problem. Note the deliberate carve-out: standardize what affects *output*, not personal ergonomics.

ITEM A5-3 · MODULE 5.2

A team wants Claude Code to follow a detailed 12-step release procedure, but only during releases — it's irrelevant the rest of the time. Best mechanism?

A. Put all 12 steps in `CLAUDE.md`

B. Package it as a skill, loaded on demand when a release task is underway

C. A hook that fires on every commit

D. Paste the procedure into the prompt each time

B. Skills exist for exactly this: task-specific procedures loaded when relevant rather than occupying context permanently. A puts 12 irrelevant steps into every unrelated session, spending context and diluting attention. C misuses an enforcement mechanism for guidance and fires at the wrong time. D works but relies on humans remembering, and drifts as the procedure evolves.

D6 Reliability, Cost & Governance

~10% · 2 modules

The "production-grade" half of the credential's scope statement — what separates a design that demos from one that can be operated, funded, and defended.

6.1 Reliability, Degradation & Observability

~75 MIN

Objectives — Design behavior for adverse conditions; instrument a system so failure is visible before users report it.

Degradation ladder — decide these before launch, not during the incident

CONDITION	DESIGNED RESPONSE
Rate limited (429)	Backoff with jitter; queue if latency-tolerant; shed load with a clear message if not.
Service overloaded / 5xx	Retry with backoff, then fall back — a smaller tier, a cached answer, or a graceful "unavailable".
Latency budget exceeded	Return partial results or a queued-work acknowledgement. Never hang.
Validation failures spike	Alarm and route to review — do not silently drop (Module 4.3).
Agent hits its budget cap	Return best-effort with the limit disclosed, or escalate. Terminate cleanly either way.
Tool dependency down	Degrade to the subset of capability that still works, and say which part is unavailable.

THE DESIGN QUESTION BEHIND ALL OF THESE

What does the user get when this system is unhealthy? An architecture with no answer will improvise one — usually a hang, a stack trace, or a confidently empty result. Deciding the degraded experience in advance is what makes a design production-grade rather than demo-grade.

What to instrument

SIGNAL	WHY IT EARNS ITS PLACE
Tokens per request/run, split by cache read/write	The cost driver, and the earliest warning of context growth or a broken cache prefix.
Agent iterations per run	Rising iteration counts precede runaway loops and cost spikes.
Validation / rejection rate	The canary for upstream change (Module 4.3).
Tool error rate by tool	Isolates a failing dependency from a failing model.
Stop-reason distribution	A rise in `max_tokens` means truncation; refusals mean scope problems.
Human gate approve/reject rate	Near-100% approval at volume means the gate is theater (Module 1.4).
End-to-end latency, p50 and p95	Agent latency is long-tailed — the mean hides the pain.
Full request payloads (redacted)	Most "the model did something weird" incidents are "we sent something weird."

NON-DETERMINISM CHANGES WHAT "TESTED" MEANS

The same input can produce different output, so a single passing run proves nothing and a single failure may not reproduce. Production readiness requires an **eval set** with aggregate pass rates, and regression testing against it on every prompt, schema, model, or tool change. A team that validated a change on one example has not tested it.

6.2 Cost Architecture & Governance Posture

~75 MIN

Objectives — Make cost a design property; establish the governance posture a production deployment needs.

THE LEVERS, RANKED — AND THEY COMPOSE

1. Right-size the model per step — the largest single lever, and in a multi-agent system it applies per participant. **2. Batch anything latency-tolerant** — half price. **3. Cache the stable prefix** — design for prefix stability (Module 3.3). **4. Control context growth** — you re-pay for history on every call. **5. Tune effort** where depth isn't buying accuracy. **6. Cap the system** — per-run and global ceilings so a bug can't produce an unbounded bill.

THE COSTS THAT SURPRISE TEAMS

Context re-sent every turn — an agent's 40th call carries 39 turns of history, so a long run's cost grows super-linearly with iterations. **Multi-agent context re-establishment** — each agent pays for its own prompt and tools. **Retries and evaluator loops** — a 3-iteration evaluator triples spend on affected records. **A broken cache prefix** — a one-character change can multiply input cost overnight with no other symptom. Model all four before launch, not after the first invoice.

Governance posture for a production deployment

1. **Data classification** — what may enter a prompt, what must be redacted, which channel is approved for each class.
2. **Credential architecture** — secrets outside model-readable context; the model sees results, never keys.
3. **Least privilege** — each agent gets the narrowest tool surface that does its job; read-untrusted separated from act-externally.
4. **Accountability** — a named human owns every consequential automated action. "The system decided" is not an answer to a regulator.
5. **Auditability** — retain enough to reconstruct why a decision was made: inputs, tool calls, outputs, approvals.
6. **Change control** — prompts, schemas, tools, and permissions are versioned and reviewed; an eval gate before rollout.
7. **Escalation path** — a defined route to a human for anything regulated, contested, or irreversible.

DESIGN DRILL 6.2 — THE READINESS REVIEW

Take your strongest design from any earlier drill and review it as if you were the approver:

- What does a user experience when the API is rate-limited? When a tool dependency is down?
- What is the per-run cost ceiling, and what enforces it?
- Which three metrics would reveal a problem before a customer does?

- If this made a costly mistake tomorrow, could you reconstruct why — and who is accountable?
- What data enters the prompt, and is that channel approved for its classification?

Any question you can't answer is a gap. **This drill is the closest single exercise to what CCAR-F actually assesses** — production-grade judgment, defended out loud, without notes.

Domain 6 — scenario items

ITEM A6-1 · MODULE 6.1

A customer-facing agent occasionally hangs for minutes when the API is overloaded; users see a spinner. What should the architecture specify?

A. A longer client timeout so requests eventually complete

B. A latency budget with a defined degraded response — partial results, a queued acknowledgement, or a clear unavailable message

C. Retry indefinitely until it succeeds

D. Switch permanently to the fastest model

B. The defect is that no degraded behavior was ever designed, so the system improvises a hang. A and C extend the hang rather than replacing it — and C can amplify an overload incident. D trades quality on every request to mitigate an occasional condition, when what's needed is a defined answer to "what does the user get when we're unhealthy?"

ITEM A6-2 · MODULE 6.2

An agentic feature's monthly cost is 6× the forecast. Iteration counts and output length look normal. Where should an architect look first?

A. Output token pricing

B. Input-side growth — cache read/write ratios and per-turn context size, since history is re-sent on every call

C. The number of users

D. Streaming overhead

B. With iterations and output normal, the overrun is on the input side: either the cache stopped hitting (a changed prefix) or context per turn is larger than modelled — and because history is re-sent every call, both compound with run length. A is ruled out by the stated evidence; C would have shown up in volume metrics; D has no cost impact at all.

ITEM A6-3 · MODULE 6.2

A team is ready to ship an agent that takes consequential customer-facing actions. Which gap should block launch?

A. No support for streaming responses

B. No way to reconstruct why a given decision was made, and no named human accountable for automated actions

C. Not using the most capable model

D. No multi-agent architecture

B. Auditability and accountability are prerequisites for consequential automated action — without them you cannot investigate an incident, answer a regulator, or assign ownership when something goes wrong. A is a UX preference; C and D are design choices that may well be correct as-is. Only B is the kind of gap that should stop a launch.

6-week study plan

Assumes CCDV-F-level API fluency. The emphasis is rehearsal, not reading — for a closed-book scenario exam, the drills are the preparation.

WEEK	MODULES	FOCUS
1	Decision frame · 1.1 – 1.2	The autonomy ladder and decomposition. Write ADRs for five real workloads.
2	1.3 – 1.5	Tool surfaces, gates, failure modes. Do the pre-mortem drill on a system you actually run.
3	2.1 – 2.5	Multi-agent. For every topology, practise arguing the single-agent counter-case.
4	3.1 – 3.4	Context architecture. Drill the four levers until the distinctions are automatic.
5	4.1 – 4.4 · 5.1 – 5.2	Contracts and Claude Code governance. Specify one contract end to end.
6	6.1 – 6.2 · full review	Reliability and cost. Run the readiness review on three designs, then re-do every scenario item.

HOW TO PREPARE FOR A CLOSED-BOOK SCENARIO EXAM

Reading is the least effective preparation. Do this instead: take any system you know, and **say aloud** which rung it sits on, what its binding constraint is, where its gates are, how its context is managed, and what happens when each part fails. Fluency under those five questions — without notes — is what the exam measures. If you can only answer them by looking something up, you're not ready.

Module tracker

Tick a module only when you can answer its question aloud, unprompted, in under a minute.

✓	MOD	TITLE	SELF-ASSESSMENT QUESTION
□	1.1	Autonomy Level	Name the four criteria. Which scenario words signal "workflow, not agent"?
□	1.2	Decomposition	What belongs to the model and what belongs to code — and why?
□	1.3	Tool Surface	State the promotion rule. What does one broad tool cost you?
□	1.4	Human-in-the-Loop	Five requirements for a real gate. Why isn't model confidence one?
□	1.5	Failure Modes	Name six modes and one structural mitigation each.
□	2.1	When Multi-Agent Wins	State the three legitimate reasons. Why isn't "specialization" one?
□	2.2	Topologies	Name six topologies and each one's characteristic failure.
□	2.3	Handoff Contracts	What must a handoff carry besides the conclusion?
□	2.4	Partial Failure	One of five parallel workers fails — what does the merge produce?
□	2.5	Multi-Agent Economics	Which topology has the worst latency profile? The worst cost profile?
□	3.1	Context Budget	What usually dominates context in an agent loop?
□	3.2	The Four Levers	Editing vs. compaction vs. memory vs. retrieval — state each distinction.
□	3.3	Caching as Constraint	Name three design choices that silently destroy prefix stability.
□	3.4	Long-Horizon Sessions	What's the difference between long-running and resumable?
□	4.1	Output Contracts	Why does a fully-required schema cause fabrication?
□	4.2	Enforcement	What does schema validity *not* guarantee?
□	4.3	Degradation	Which failure strategy is always wrong, and why?
□	4.4	Contract Evolution	How do you add a required field without breaking consumers?
□	5.1	Configuration Layers	Which layers are enforced and which advisory? Where does "must never" go?
□	5.2	Team Governance	Name the two governance failure modes and their fixes.
□	6.1	Reliability	What does a user get when the system is unhealthy? Name five metrics.
□	6.2	Cost & Governance	Rank the cost levers. Name the four costs that surprise teams.

ANSWER KEY — QUICK REFERENCE

D1: 1-B · 2-B · 3-C · 4-C · **D2:** 1-B · 2-B · 3-B · 4-B · 5-B · **D3:** 1-B · 2-C · 3-B · 4-B · **D4:** 1-B · 2-B · 3-B · 4-B · **D5:** 1-C · 2-B · 3-B · **D6:** 1-B · 2-B · 3-B

The answer letters cluster because correct architectural answers share a shape: name the constraint, choose the structural control, keep the human accountable for the irreversible step. **Don't pattern-match the letter — rehearse the reasoning.** On the real exam the options are shuffled, and the rationale is the only thing that transfers.